USER GUIDE + TECHNICAL EXPLANATION

NeuroLog AI
Complete handbook

Learn every tab, follow a log from its source to the dashboard, understand the analysis, and present a confident demonstration.

Current version: the application-first workspace with the combined Investigations page. Checked against the current source code on 6 October 2026.

Who this is for

Read this without knowing programming. The early chapters explain the interface in simple language. Later chapters explain the code, logging integration, algorithms, hosting, troubleshooting, and limitations.

The explanation to remember

Log explorer tells us what happened. Investigations helps us understand what it means. The assistant helps us ask questions about it.

Live app: neurolog-ai.onrender.com
Repository: shahrukh-210906/Neurolog-AI

This guide distinguishes actual activity, generated scenarios, deterministic local analysis, and optional language-model responses.

LEARN THE APPLICATION

Your reading map

Pages	Topics
3-5	Purpose, vocabulary, architecture
6-9	Applications, Overview, Log explorer, Investigations
10-16	Patterns, clusters, evidence, risk, message space, alerts
17-21	Screenshot, assistant, integrations, contract, Settings
22-24	Python source, Guest Connect, idle logs and outcomes
25-27	How we built it, code map, storage and APIs
28-30	Local startup, Render, troubleshooting
31-33	Demo script, common questions, production roadmap
34-36	References and reusable integration examples

A useful learning order

Understand the main tabs first. Then perform one real action in Guest Connect and find its outcome in NeuroLog. Use separately labeled generated scenarios to learn error analysis. Read the implementation chapters before answering technical questions.

Incidents and Patterns & Outliers used to be separate pages. They are now Investigations. Old /incidents and /analysis links redirect there. Pipeline Lab has been removed from navigation.

Older repository documents and MongoDB scripts describe previous versions. This guide follows the maintained Flask/SQLite backend and current React interface.

1. What the application does

Applications perform operations: opening pages, saving events, handling requests, connecting to databases, and sometimes failing. A log is a written record of one of those activities.

NeuroLog receives these records and makes them easier to understand. Search raw activity, find repeating messages, group similar wording, highlight unusual messages, and review related errors together.

A concrete example

Create an event in Guest Connect. It records the action and result, then sends sanitized activity to NeuroLog. Select guest-connect, open Log explorer, and search event_created or save_succeeded. An outcome is more useful than a click alone.

What it adds

- One place for logs, with a separate view for each selected application.

- Less manual reading: repetition becomes named patterns, and related warning/error records become investigation groups.

- More context: source, severity, record ID, receipt time, entity, trace, recurrence, and analysis evidence.

- An assistant summarizes received evidence and suggests checks. You verify the conclusions.

Describe it as a log collection and investigation application. It does not automatically fix bugs, prove root causes, capture literally every operation, or replace every mature observability platform.

LEARN THE APPLICATION

2. Understand the vocabulary

Term	Meaning
Log / record	One received message with metadata and a record ID.
Source	A sender label, such as guest-connect or task-service.
Severity	The seriousness reported by the sender.
Pattern	A normalized message shape that repeats.
Cluster	Messages with similar wording, grouped by clustering.
Outlier	A message isolated by the last clustering run.
Incident group	Related warning/error evidence grouped for review.
Entity	A resource label; source is the fallback.
Trace	A label connecting activity from one operation.
Risk	A heuristic review priority, not a failure probability.
Ingestion key	A secret used to submit logs.
Outbox	A queue holding events until accepted.

A heartbeat may repeat all day and form a pattern without being an error. Repeated database errors may form a pattern and contribute to an incident. They overlap, which is why the UI brings them together.

3. The system at a glance

The supported stack has a React/Vite dashboard, a Python Flask API, SQLite storage, and applications that send logs. The Python task service and Guest Connect demonstrate real integrations.

Layer	Responsibility
Sender	Records operations or explicitly generated scenarios.
Outbox	Buffers events and retries unsuccessful delivery.
Ingestion API	Checks the key and fields, assigns receipt time, stores the record.
SQLite	Stores records, keys, settings, rules, and analysis metadata.
Analysis	Finds repetition, categories, risk, clusters, and neighbors.
Dashboard	Polls the API and displays the selected source.
Assistant	Local rule analysis or optional Groq response.

```
Application operation -> logger/activity event
  -> outbox -> POST /api/ingest -> SQLite
  -> analysis endpoints -> React dashboard
```

The interface uses HTTP polling, not WebSockets. Delivery is near real time, with delay from batching, retries, and page refresh. Typical pages refresh every three or five seconds.

4. Applications: choose the workspace

Say: "This is where we choose the application we want to monitor."

The app opens the application picker first. A source appears after at least one log is accepted. Creating a key alone does not prove that an external application is running.

Use it

- Find guest-connect or task-service; search by name if needed.
- Check received-record count and last-received timestamp.
- Open the application. The main views now request that source.
- Use Switch app to select another source.

How selection works

The frontend stores the selection in current page-session state and adds source to relevant API requests. The backend filters source before applying the record limit. Switching applications remounts views so old state is not mixed.

Recently reporting means receipt within roughly two minutes. It is not a guarantee of uptime: a quiet app can be healthy, and a failing app can still send logs.

A full reload starts at the picker again. Select the correct source before reviewing activity, especially when Guest Connect and the Python service are both reporting.

5. Overview: the quick summary

Say: "This gives a quick picture of the selected application and what deserves attention."

Card	Meaning
Records received	Up to the latest 300 records, not lifetime total.
Errors to review	Records with syslog severity 0 through 3.
Clustering outliers	Stored flags from earlier analysis.
Latest receipt	When the newest record was received.

The chart groups received records by minute, separating routine from error activity. Connection information shows receipt and generated-origin counts. Named patterns summarize repetition. Recent errors and quick links provide investigation starting points.

For task-service, Error scenarios starts a bounded synthetic workload through the real ingestion pipeline. These scenarios do not prove Guest Connect or an actual database is failing.

Zero errors does not mean zero bugs

It means none were received in the current window. Errors could be older, uninstrumented, queued, filtered, or from another selected application. API connected means the dashboard can reach NeuroLog, not that every external service is healthy.

6. Log explorer: original evidence

Say: "This is the detailed activity list. We can search it to find what was recorded."

- All activity, Errors & critical, Warnings, and Routine filter severity.
- Search matches message, source, entity, and trace in the current window.
- Origin separates actual activity from messages starting [GENERATED].
- Pause stream stops automatic display refresh, not ingestion.
- Reset clears filters when records seem missing.

Inspect a log

The inspect icon opens the full record: ID, source, receipt time, origin, entity/trace, message, and last cluster result. Copy the message or open the assistant with source context.

Try Guest Connect

Select guest-connect, create/update an event, and search event_created, event_updated, save_succeeded, or save_failed. A click or save_started record proves an attempt; a successful outcome is separate evidence.

Explorer fetches up to 300 records every three seconds. Search filters that window, not all history. Receipt time is assigned by NeuroLog and can be later than the original operation if delivery was delayed.

7. Investigations: one place to investigate

Say: "Here we bring repeating activity, possible incidents, and supporting evidence together."

View	Question it answers
Overview	Which related errors deserve review?
Named patterns	What messages repeat?
Clusters	Which messages have similar wording?
Outliers	Which analyzed messages are isolated?
Evidence ledger	What records support the finding?
Message space	Which messages are nearest in text-vector terms?
Alert rules	Which conditions should draw attention?

The selected view is in the URL, supporting browser Back and Forward. Pause controls automatic refresh; it does not stop senders. Explicit analysis actions remain available.

Raw error alerts counts error/critical records. Grouped incidents includes warning/error groups. Their populations differ, so Grouping reduction is a bounded heuristic, not a measured accuracy or time-saving score. Analysis time measures server computation, not total UI latency.

Most evidence uses a bounded recent window. Recurrence and correlation checks specifically use the last five minutes.

8. Named patterns: remember repetition

Say: "Repeated messages get a memorable name so we recognize the behavior quickly."

Changing numeric measurements and UUID-like IDs are normalized. Records are grouped by source, generated/actual origin, severity, and template. At least two matching records are needed.

```
Database connection timeout duration_ms=125
Database connection timeout duration_ms=480
  -> Database connection timeout duration_ms=<n>
```

Each card shows name, severity, count, source, origin, and share of the current window. Expand Message & investigation for the template, sample evidence IDs, first/latest receipt, and next checks. Up to eight patterns are displayed.

Names are suggestions

Keyword rules produce labels such as Database timeouts, Memory pressure, Upstream failures, Rate-limit bursts, and Authentication failures. Fallback names come from routine wording or the template. There is no custom naming editor.

Rules can overmatch. A healthy heartbeat containing database=memory may receive Database timeouts because of the database keyword. Read the actual message and severity; a label is not a diagnosis.

Patterns refresh about every five seconds. Repeated INFO activity can be normal; repetition alone is not an incident.

9. Clusters: group similar wording

Say: "Messages with similar wording are grouped together to reduce manual reading."

Use Run analysis, Run clustering, or the Settings action. The backend currently analyzes up to the newest 100 selected-source logs, with at least five required. Views can show 300 records, so older displayed flags may come from previous runs.

TF-IDF in simple language

Words become numerical weights. Common words contribute less; distinctive terms help separate message types. The implementation normalizes changing numbers and IDs before vectorizing.

DBSCAN in simple language

It finds groups of nearby message vectors without requiring you to choose a number of groups. Current settings are eps=0.5 and min_samples=2, using the library default distance metric. Cluster IDs are machine group numbers, not stable incident identities.

Use the result

Review the example message and source for a group, then open Log explorer to inspect evidence. Group membership describes wording, not a proven shared root cause.

Clustering runs only when requested. Automatic pattern detection is separate. Running either process does not train a general-purpose language model.

10. Outliers: investigate unusual messages

Say: "These messages do not fit the groups from the last analysis, so we review them."

DBSCAN marks ungrouped points with cluster ID -1. The API stores an outlier flag on those records. The Outliers view lists flagged records and links to the assistant.

What an outlier can mean

- A genuinely unusual error message.
- A rare but successful operation.
- New wording after a code change.
- Too few similar records in the analysis window.

What to do next

Read the original message, severity, source, surrounding activity, and operation outcome. Compare it with known normal behavior. Ask for verification steps rather than assuming the application is failing.

A zero outlier count can be a valid result. It does not prove the application is perfect. Results describe the latest stored clustering state, and the display window can include records from earlier runs.

Outlier status contributes 10 points to the heuristic risk score. It is not a confirmed incident label, uptime metric, security verdict, or automatic remediation trigger.

11. Evidence ledger and record replay

Say: "This is the investigation table, including why each record received its risk score."

Search message, trace, source, or entity. Filter category and risk: high is 65+, medium 35-64, and low below 35. Grouped view shows one representative per signature; Raw view shows each record.

Export CSV exports all matching rows, even when grouped view displays fewer representatives.

Inspect processing evidence

Replay shows ingestion details, available vector terms, stored cluster, category, risk factors, recurrence, and next checks. It does not re-execute the original request. Select SRE, beginner, or structured JSON explanation format.

Click an entity for its history within the current application window. If no entity ID was supplied, source is the fallback. This is not unrestricted cross-application tracing.

Suppress a false positive

Suppression subtracts 45 risk points from that signature, bounded at zero. It preserves raw evidence, can be reversed, and does not retrain a model or necessarily hide the group.

The signature includes source, normalized wording, and generated/actual origin. Generated examples remain distinct from actual application behavior.

12. Risk and incident grouping

Risk is a transparent review priority, not the probability of failure. Scores sum the factors below and are clamped to 0-100.

Factor	Points
Severity base	Emergency 80; Alert 75; Critical 70; Error 50; Warning 25; Notice 10; Info 5; Debug 0.
Recent recurrence	+20 for warning/error with at least three same-category/entity/origin events in five minutes.
Stored outlier	+10 when the clustering flag is set.
Suppressed signature	-45 for the analyst preference.

Example: an ERROR starts at 50, recurrence raises it to 70, and an outlier raises it to 80. Risk above 80 is strictly greater than 80, so a score of exactly 80 does not match.

Incident groups collect warning/error records by category, entity, and origin. Each group stores count, maximum risk, sources, evidence, and advice. Group totals use the record window; recurrence bonuses use five-minute timing.

Overview highlights recent repeated database, upstream, and authentication warning/error categories when at least three occur. Shared timing and category are correlation, not proven dependency causation. There is no assignment or resolved-status workflow yet.

13. Message space: visual similarity

Say: "Each point represents a message. We can inspect it and find similar wording."

This view uses up to the latest 150 selected-source records, TF-IDF with up to 1,500 terms, and a two-dimensional truncated SVD projection when there is enough variation.

- Select a point, or use the accessible record dropdown.
- Review up to five nearest messages using cosine similarity in the original vector space.
- Inspect weighted terms and other record evidence in the replay drawer.

Interpret it correctly

It is a message-wording map, not server geography, request routing, uptime, or a dependency graph. Colors show rule-based categories. Two nearby points are a clue, not proof of the same problem.

Projection does not preserve every distance. The numeric cosine similarities come from the original TF-IDF vectors, making them more direct evidence of wording similarity than screen distance.

Projection is distinct from stored DBSCAN clustering. Identical or limited-vocabulary messages use a simpler coordinate fallback. No usable vocabulary produces an empty result rather than invented points.

14. Alert rules: define attention

Say: "We define conditions that should draw attention when matching evidence appears."

Condition	Behavior
Risk above 80	Group maximum risk is strictly greater than 80.
Database warning / error	Database-category incident groups.
Repeated authentication rejection	Auth group containing a record escalated by recurrence.

Choose a condition and action, Add rule, inspect matched payloads, and remove unused rules. Rules are scoped to the selected source, with a limit of 20 across the workspace.

In-app versus external actions

In-app notification creates a notice for a newly observed match identity. PagerDuty, Slack, and Webhook create reviewable payload previews; no external message is dispatched.

Current behavior to explain

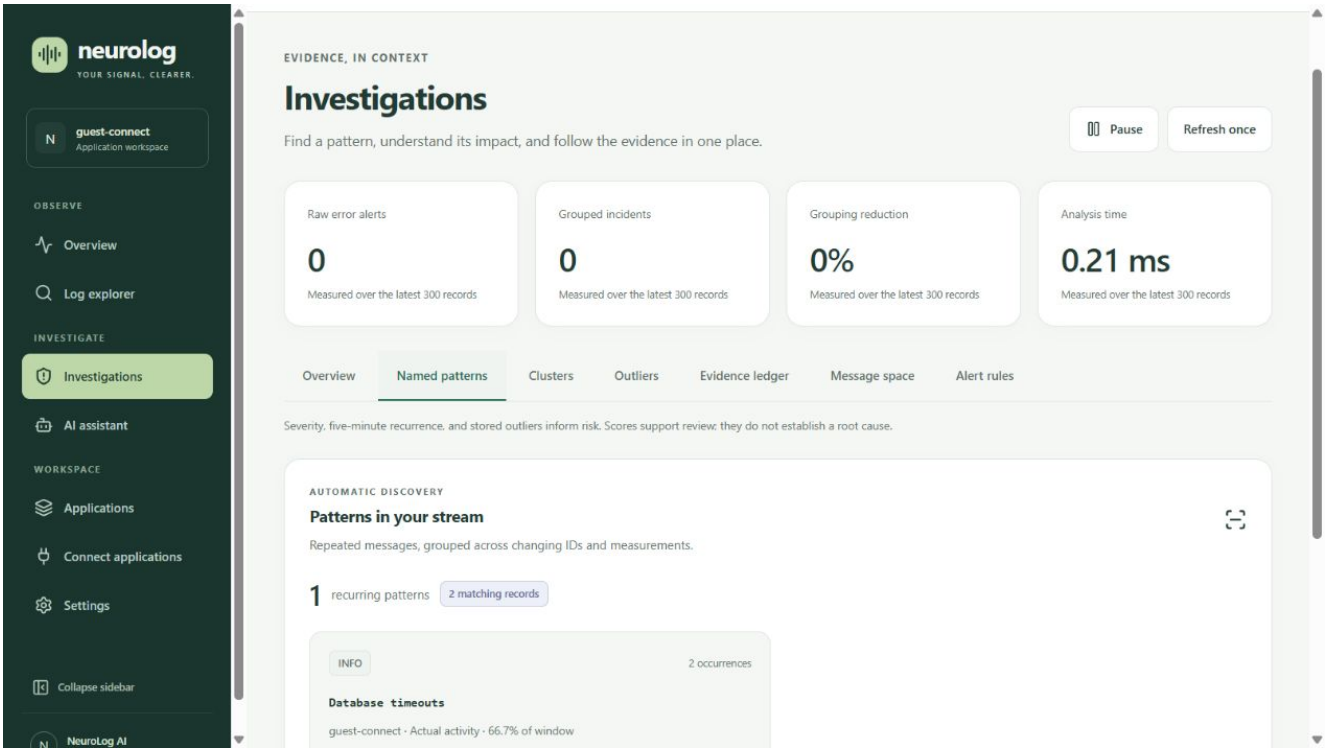
Rules are evaluated when the intelligence endpoint is requested, rather than by an independent always-on scheduler. New-match identities are remembered in memory while the page is mounted. Reloads may surface a match again; every extra record is not necessarily a new identity.

Generated scenarios can match rules but remain separately identified. They demonstrate the rule, not a real outage. No email, SMS, or on-call delivery is enabled by choosing a preview action.

CURRENT WORKSPACE

15. Investigation screenshot

The combined workspace keeps the application context visible and all investigation views in one place.



This screenshot illustrates layout and navigation. Counts, timestamps, and names are a moment-in-time example, not fixed data. Always verify the original evidence behind an automatically suggested name.

16. AI assistant: ask about evidence

Say: "We ask questions about the logs and receive explanations and next checks."

Local evidence analysis

Without a provider key, deterministic rules group warnings/errors and stored outliers, summarize severity, identify repetition, cite record IDs, and suggest checks. This is not a language-model response and does not understand every arbitrary question like a chatbot.

Optional provider mode

GROQ_API_KEY enables Groq on the backend. GROQ_MODEL can override the configured default. Provider failures fall back to local analysis with a notice. The UI identifies the mode.

The backend supplies up to 300 selected-source records; provider context limits each message to 800 characters. It sends the current question and logs, not prior conversation turns as model history.

Use it

- "Find repeated patterns source:guest-connect and cite evidence."
- "Summarize recent errors and suggest verification steps."
- "Explain recent anomalies and clustering results."

Displayed history is separate per source in sessionStorage, saving the last 50 messages. Enter sends; Shift + Enter adds a line. Clear conversation clears that displayed history. Neither mode executes fixes. Provider mode sends log context to the configured external provider.

17. Connect applications: real integration

Say: "This provides the endpoint, key, and example code for another application."

- Use a stable source name and create an ingestion key.
- Save the full key privately when shown; listings later show metadata and suffix.
- Choose Python, Node.js, or cURL; configure the sender endpoint and key.
- Send a connection test, then perform a real external-app operation.
- Verify its specific record, then open its application workspace.

A UI test proves that test path works. It does not prove the other application has correct environment variables, network access, or business-event instrumentation.

The Python download uses a durable SQLite outbox. The minimal Node example uses timeout/error handling but has no durable queue. Guest Connect uses its own disk-backed sender.

Revoking a key stops new submissions using that key while retaining old logs. Source labels are caller supplied: a key name does not enforce one immutable application source.

Store keys server-side, never in browser bundles or Git. A successful send returns HTTP 201. Any language can use the JSON ingestion contract.

18. The ingestion contract

Hosted endpoint: **https://neurolog-ai.onrender.com/api/ingest**. Send POST, Content-Type: application/json, and the private x-api-key header.

Field	Contract
message	Required text; up to 10,000 characters.
source	Optional label; up to 120 characters; defaults to external-app.
severity_level	Optional integer 0-7; default INFO (6).
entity_id / trace_id	Optional labels; up to 120 characters each.
timestamp	Assigned on receipt, not supplied by sender.

```
POST /api/ingest
Content-Type: application/json
x-api-key: YOUR_PRIVATE_KEY

{
  "source": "my-application",
  "severity_level": 6,
  "message": "Event save completed",
  "entity_id": "event-service",
  "trace_id": "operation-123"
}
```

201 means accepted; 401 means missing, invalid, or revoked key. Invalid fields return validation errors. Keep sensitive values out of messages; this API is not a universal redaction system.

19. Settings and shared controls

Say: "Settings adjusts appearance and explains analysis and connection setup."

Appearance

Light, Dark, or System is saved on this device. System follows the operating system. The top-bar theme icon offers a quick light/dark switch.

Analysis, AI, and source

Run clustering refreshes stored analysis. Select an application first for a scoped demo; Settings is accessible without a selection. Provider setup uses backend environment variables, not a key editor in this card. Connected source shows selected context and API reachability, with a link to integrations.

Useful controls

- Switch app returns to the picker; global search opens Explorer.

- Ctrl + K focuses global search; Run analysis refreshes clustering.

- Investigations: Space pauses/resumes and N requests refresh when focus is outside interactive controls. Escape closes drawers.

Key management is in Connect applications. The current Settings UI does not expose the backend retention or deletion APIs as ordinary visible settings. API reachable is not independent sender-health verification.

20. Python source and generated scenarios

The built-in Python application creates, completes, and reopens tasks. Local URL: http://127.0.0.1:8000. Hosted path: /source/. Actual task operations and HTTP outcomes emit real logs; tasks persist locally in SQLite.

Requests log method, path, status, and measured duration. Heartbeats record measured runtime information. A background logging handler queues delivery so it does not need to block each operation.

The opt-in generator

Intervals are 0.2-10 seconds, with a limit of 1-1,000 entries. It stops at the limit or when stopped. Every scenario begins [GENERATED]. Its error types follow a repeating cycle; numeric measurements are randomized.

Types include successful requests, background jobs, rate limits, database timeouts, upstream failures, memory errors, and authentication failures. These messages do not intentionally break real dependencies or exhaust memory.

Delivery reliability

The SQLite outbox holds up to 10,000 events and removes them after acceptance. Retries back off up to 30 seconds. Queues survive sender restarts when the disk persists. A lost acknowledgment can create duplicates: at least once delivery, not exactly once.

The task-service generator is separate from Guest Connect. Demonstrate actual activity and synthetic errors as clearly labeled, separate examples.

21. Guest Connect: what is connected

Guest Connect sends sanitized browser/server activity as guest-connect. Its Express server holds NEUROLOG_ENDPOINT and NEUROLOG_INGEST_KEY in an ignored environment file. The browser never receives the NeuroLog key.

Backend events

Fixed-route request logs include method, status, duration, and request ID. Startup, 30-second heartbeat, database outcomes, generic failures, and scheduled reminder failures can also appear. Telemetry receiver requests are excluded from routine request logging to avoid loops.

Browser events

Both demo and live modes report navigation, visibility, button actions, field-edit categories, debounced searches, filters, selection/dialog changes, validation, saves, guest/event create/update, assignments, transport, checklists, invitations, exports, print workflow, and runtime failure categories.

Browser events use a short-lived session and same-origin endpoint. The server validates event/page/target enums, drops extra fields, and places accepted events in a disk-backed delivery queue.

Limits and privacy

Raw typed values, names/contact details, invitation bodies, headers/bodies, query strings, raw exceptions, mouse movement, and raw keystrokes are excluded. Demo-mode activity is logged but remains browser-only business state. Live mode additionally produces backend save outcomes.

22. Idle logs and save verification

Why logs appear without clicks

Heartbeats, startup, visibility changes, background requests, and scheduled work are real runtime activity. They are not evidence that a random generator is running. Check source and the [GENERATED] origin marker.

Prove that an event save is recorded

- Select guest-connect, clear Explorer filters, and create/update an event using development data.
- Allow batching, delivery, and polling time.
- Search event_created/event_updated and save_succeeded, or save_failed for a failed operation.
- Inspect record IDs and receipt times. A click or save_started proves only the attempt.

Buffering limits

Browser buffering holds up to 500 unsent sanitized events in memory, batches up to 40 every two seconds, and may lose pending events when the tab closes. Accepted backend events are durable on persistent disk. Telemetry is limited to 240 events per minute per session.

Print completion means the dialog closed, not that paper printed. Export start is not export completion. Connection badges are not proof of business-event receipt. Verify a specific outcome.

23. How we built and improved it

We started from the original MIT-licensed repository and retained attribution. The maintained stack uses React/Vite, Flask, SQLite, and scikit-learn; older MongoDB scripts remain outside the supported runtime.

- Established working startup, ingestion validation, storage, and a login-free presentation flow.

- Added a real Python task app and durable shipping, followed by labeled error scenarios.

- Implemented named repetition, local explanations, clustering, heuristic risk, projection, record replay, and alert previews.

- Redesigned the workspace with calmer navigation, preserving the supplied visual reference direction.

- Added guided integrations, application-first selection, scoped queries, and separate displayed assistant history.

- Connected Guest Connect and instrumented sanitized browser/server outcomes in demo and live modes.

- Combined incident and pattern views into Investigations, retained old-route redirects, and removed Pipeline Lab.

This combines frontend engineering, APIs, storage, background delivery, text analysis, and application instrumentation. It is not a newly trained foundation model.

24. The code map

File / area	Responsibility
src/App.jsx	Routes, selected application, lazy pages, global actions.
src/components/Sidebar.jsx	Main navigation; one Investigations entry.
src/api.js	HTTP client and source parameters.
pages/Applications.jsx	Application picker.
pages/LogIntelDashboard.jsx	Summary metrics and activity chart.
pages/LogExplorer.jsx	Raw records, filtering, detail drawer.
pages/Incidents.jsx	Combined investigations, ledger, replay, rules, vectors.
pages/VectorAnalysis.jsx	Embedded clusters and outliers.
components/PatternPanel.jsx	Recurring named cards.
pages/AIAssistant.jsx	Question UI and displayed source history.
pages/Integrations.jsx	Keys, examples, and connection tests.
LogIntel_engine/api.py	API, SQLite, validation, clustering, chat.
analysis.py / intelligence.py	Normalization, names, categories, risk, groups, vectors.
live_app/	Task app, durable handler, generator.
render_app.py / render_wsgi.py	Unified hosted application.

Frontend paths above are under logintel_ui/src. Incidents.jsx retains its historical filename although the page is now Investigations. Legacy server/ and MongoDB scripts are not the maintained Render runtime.

25. Storage, APIs, and refresh behavior

Local data includes neurolog-live.db for received records/API metadata, tasks.db for tasks, and task-log-outbox.db for Python delivery. Key digests use SHA-256; full keys appear at creation. Data and key files are excluded from Git.

Endpoint	Purpose
GET /api/health	API/database reachability.
GET /api/connections	Source totals and last receipt.
POST /api/ingest	Key validation and record storage.
GET /api/recent-logs	Bounded source-scoped records.
GET /api/patterns	Recurring templates and names.
GET /api/intelligence	Risk, groups, correlations, rule matches.
GET /api/vector-space	Projection, weighted terms, neighbors.
POST /api/run-ml	Store DBSCAN results.
POST /api/chat	Local or optional provider explanation.
/api/alert-rules	List/add/remove scoped rules.

Overview and Explorer poll every three seconds. Picker, patterns, investigations, and assistant evidence generally poll every five seconds. Clustering is explicit. Retention defaults to 14 days; older records are cleaned up when recent records are requested. The API supports 3, 14, or 90 days, but Settings has no retention editor.

26. Run locally and validate

Use Python and Node.js. The launcher creates the environment when absent, installs locked Python dependencies and frontend packages when needed, then starts hidden background services.

```
powershell -ExecutionPolicy Bypass -File .\start-preview.ps1

# Restart managed services after backend changes:
powershell -ExecutionPolicy Bypass -File .\start-preview.ps1 -Restart
```

Service	Local URL
Dashboard	http://127.0.0.1:5173
API health	http://127.0.0.1:5001/api/health
Python task app	http://127.0.0.1:8000
Guest Connect (separate)	http://127.0.0.1:3001

Create a task, select task-service, and find its log. Guest Connect starts separately from its own project. Avoid multiple senders sharing one outbox.

```
.venv/Scripts/python.exe -m pytest -q
npm --prefix logintel_ui run lint
npm --prefix logintel_ui run build
```

The recent backend suite passed 32 tests; Guest integration passed 10 tests. The Investigations merge passed lint/build and browser checks. These are point-in-time checks, not proof of zero possible bugs.

27. Render hosting and updates

The hosted app is one Docker web service built from codex/neurolog-fixes. Docker compiles React and packages Python. Gunicorn uses one worker with four threads and binds to Render PORT.

The same origin serves UI and /api; the Python source is mounted at /source/. Hosted task events use an internal transport through ingestion validation, with a background measured heartbeat.

Update flow

Verify locally, commit, push the configured branch, and manually deploy the latest commit. Check service-live status, API health, actual source receipt, and changed views. Build success alone does not prove integration works.

Storage and keys

Hosted filesystem storage is ephemeral. Restarts/redeploys can reset logs, tasks, queues, and keys. A sender may need a replacement key configured privately and a restart. Use persistent storage or a managed database for an archive.

A free instance can take time to wake. Process shutdown stops the generator. Do not silently restart scenarios while presenting actual application activity.

Public demonstration access

This is intentionally login-free. Public visitors can access workspace functions. Authentication, roles, durable storage, and operational protection are needed before confidential production use.

28. Troubleshooting

Symptom	Check
No application card	Source needs an accepted first record; verify sender and HTTP 201.
Missing event log	Select guest-connect, clear filters, save, search outcomes, allow batching/retry.
Idle records	Read source/origin; heartbeats and background work are real activity.
Zero errors	Healthy recent window, wrong source, filtering, or queued/uninstrumented errors.
No clusters	Need five records, usable words, and an explicit analysis run.
Zero outliers	Can be a valid result; inspect the last stored run.
Misleading pattern name	Read original wording and severity; keywords can overmatch.
401 delivery response	Check missing/revoked key or Render reset; replace privately.
Rule-based assistant	Provider key unset or provider failed; read mode notice.
Local startup failure	Check occupied ports and data/*.stderr.log.

After API recovery, queued delivery can take the retry-backoff interval plus UI refresh. Sender durability cannot make an ephemeral hosted disk durable. Do not delete evidence merely to hide a symptom.

29. Present a five-minute demo

Prepare Guest Connect and NeuroLog, confirm receipt, and ensure no unwanted generator is running. Use non-sensitive development data.

- **0:00 Purpose:** "Applications produce logs. NeuroLog helps us collect and understand them."
- **0:30 Selection:** open guest-connect. "Each application has its own workspace."
- **1:00 Real action:** create/update an event. "We verify the save outcome, not just the click."
- **1:30 Explorer:** search event_created/event_updated and save_succeeded; inspect ID, source, time, severity.
- **2:15 Investigations:** show repetition and evidence. "A pattern can be normal; related errors deserve review."
- **3:00 Optional scenarios:** switch to task-service, explicitly start labeled errors, collect enough records, run clustering.
- **4:00 Assistant:** ask for evidence and next checks; show local/provider mode.
- **4:40 Wrap up:** "We turn activity into searchable evidence and guided investigation. We do not automatically fix services."

Stop the scenario generator afterward. If there are no actual errors, say so. Use labeled scenarios instead of implying healthy activity proves an outage.

30. Questions others may ask

Is it really connected?

"We perform a real action and find its outcome in NeuroLog. A test message or badge alone is insufficient."

Is it random?

"Guest Connect sends real instrumented activity. The optional Python generator labels synthetic events. Idle heartbeats are actual measurements."

Are patterns and incidents the same?

"Patterns describe repetition. Incident groups collect related warning/error evidence. They overlap, so they share Investigations."

Where is the AI?

"TF-IDF/DBSCAN analyze wording; rules explain evidence and risk; optional Groq provides conversational responses. We did not train a new language model."

Can it fix problems or beat every logging platform?

"It suggests checks; it does not execute repairs. This focused project supports application-first investigation. Mature platforms can offer broader search, tracing, security, alert delivery, and production operations."

Does ERROR mean an outage?

"It means the sender reported an error. We still inspect scope, outcome, surrounding activity, and independent measurements."

31. Limits and the production roadmap

- **Persistence:** durable storage, backups, and migrations.
- **Access:** authentication, roles, tenant isolation, protected administration, and rate limits.
- **Source trust:** bind allowed source labels to keys when required.
- **Accuracy:** improve keyword naming and evaluate with labeled evidence.
- **History:** pagination and time-range queries instead of only bounded windows.
- **Delivery:** event IDs/idempotency, original event time, queue health, and latency measurement.
- **Alerts:** background evaluation and authenticated external delivery.
- **Incidents:** ownership, acknowledgment, and resolution states.
- **Privacy:** redaction, retention controls, auditing, and provider-consent design.

These are future improvements, not existing features. Current risk is heuristic, clustering describes wording, projection is approximate, and assistant conclusions need verification. UI source scoping does not provide production-grade security on a public login-free workspace.

32. References and quick reference

This handbook was checked against current source code. Implementation takes precedence when older documents differ.

Primary references

React pages and components under logintel_ui/src; LogIntel_engine/api.py, analysis.py, intelligence.py; live_app/app.py, generator.py, log_shipper.py; render_app.py, render_wsgi.py; Dockerfile; start-preview.ps1.

Guest Connect: server/neurolog.js, server/activity.js, src/activity.js, src/activitySchema.js, and activity/save hooks in src/main.jsx. Earlier NEUROLOG.md prose saying demo activity is unmonitored is superseded by detailed browser instrumentation.

Supporting docs: INVESTIGATIONS.md, APPLICATION_WORKSPACES.md, CONNECT_APPLICATIONS.md, RENDER_DEPLOYMENT.md. Earlier guides contain historical tab names, intervals, and test counts.

Original MIT project: Sai-Nitin123/Neurolog-AI.
Current repository: shahrukh-210906/Neurolog-AI.

Remember the main tabs

Applications: choose. **Overview:** summarize. **Explorer:** search. **Investigations:** understand. **Assistant:** ask. **Connect applications:** integrate. **Settings:** customize.

Demo rule: show source, original record, and outcome before explaining analysis. Label generated activity and distinguish evidence from hypotheses.

REUSABLE INTEGRATION EXAMPLE

Appendix A. Connect a Python logger

Download neurolog_handler.py from Connect applications into your application folder. Create a server-side ingestion key and set NEUROLOG_INGEST_KEY in the sender environment. Keep the key out of source code.

```
import logging, os
from neurolog_handler import NeuroLogHandler

handler = NeuroLogHandler(
    endpoint="https://neurolog-ai.onrender.com/api/ingest",
    source="my-python-app",
    spool_path="data/neurolog-outbox.db",
    key_path="data/neurolog.key",
    api_key=os.environ["NEUROLOG_INGEST_KEY"],
)
handler.start()
logger = logging.getLogger("my-python-app")
logger.setLevel(logging.INFO)
logger.addHandler(handler)

# Call after a real operation succeeds:
logger.info("Event save completed", extra={
    "entity_id": "event-service",
    "trace_id": "operation-123",
})

# At application shutdown:
handler.close()
```

Set a stable source. Use actual operation correlation IDs when available; operation-123 above is an illustrative value. Log WARNING or ERROR inside genuine rejection/failure paths, not to manufacture an outage.

Python CRITICAL maps to syslog 2, ERROR to 3, WARNING to 4, INFO to 6, and DEBUG to 7. Give separate applications separate queue files. Remote delivery requires an explicitly configured key; automatic provisioning is restricted to localhost.

MINIMAL INTEGRATION EXAMPLE

Appendix B. Connect a Node application

This example requires a Node runtime with built-in fetch and AbortSignal.timeout. It demonstrates the ingestion contract but has no durable queue. Guest Connect already has a stronger disk-backed sender.

```
async function sendLog(message, severity = 6) {
  const response = await fetch(
    "https://neurolog-ai.onrender.com/api/ingest", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "x-api-key": process.env.NEUROLOG_INGEST_KEY,
      },
      body: JSON.stringify({
        source: "my-node-app",
        message,
        severity_level: severity,
      }),
      signal: AbortSignal.timeout(5000),
    }
  );
  if (!response.ok) {
    throw new Error(`Log delivery status ${response.status}`);
  }
}

// After an actual successful operation:
void sendLog("Event save completed").catch(() => {
  console.warn("Telemetry delivery failed");
});
```

Logging delivery failure should not corrupt the original business operation. A production sender should buffer events durably, retry with bounded backoff, expose queue health, and avoid recursively shipping its own delivery failures.

For Guest Connect, update its existing NEUROLOG_ENDPOINT and NEUROLOG_INGEST_KEY privately, then restart its backend. Its business API_KEY is a different credential. Do not add this minimal sender alongside the existing one and accidentally duplicate instrumentation.